

Ascend 950 组网架构与 SHMEM/HIXL单边通信技术解析

<https://gitcode.com/cann>

CANN

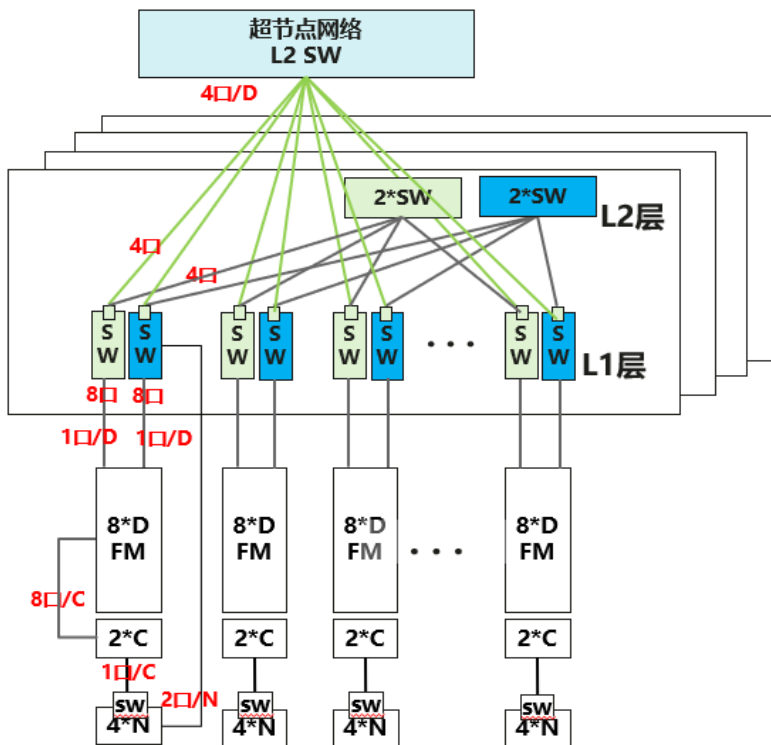
目录

Part 1 Ascend 950组网架构

Part 2 HIXL编程API介绍

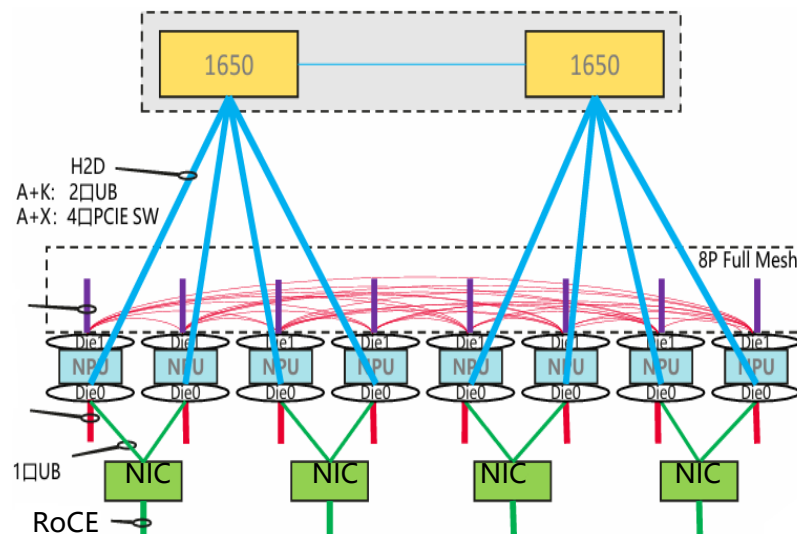
Part 3 SHMEM AIV直驱URMA

Ascend 950 硬件形态：POD & Server & 标卡



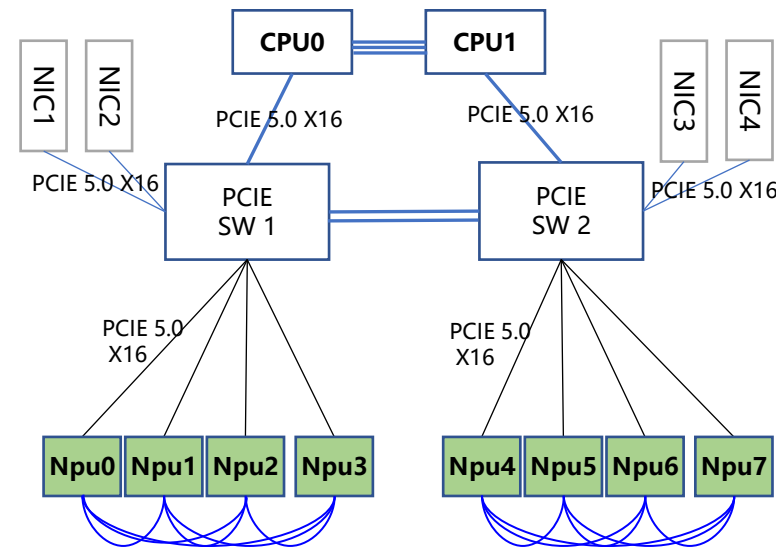
POD形态

- ① 1D fullmesh+clos拓扑, 64 NPU + 16 CPU
- ② 计算板：每D出7*X4 UB 端口组成 Fullmesh连接, 每D出8口对接L1 SW
- ③ 交换板：通过不同的L2 SW配置, 可以等效支持每D出4个端口或者8端口, 作为框内板间互联或者框间互联



Server形态

- ① 8 NPU + 2 CPU, CPU节点支持鲲鹏及X86
- ② server内：Fullmesh拓扑, 7*X4 UB口
- ③ 出server：Clos拓扑, 8*X4 UB口



标卡形态

- ① 基于X86底座, 支持四卡UB互联
- ② 每卡3个X4 UB端口组成Fullmesh连接
- ③ 可接第三方网卡, 做参数面互联方案

Ascend 950超节点采用POD& Server作为基本互联单位，通过灵衢互联构建高性能集群

- 1、超节点内CPU、NPU、存储等异构组件采用无主从直接通信，实现组件对等协同；
- 2、超节点内协议归一，所有节点都使用一套协议互连互通；
- 3、超节点内原生支持内存语义，兼顾异步访问的消息语义，既保证了极致性能，又兼顾兼容性；

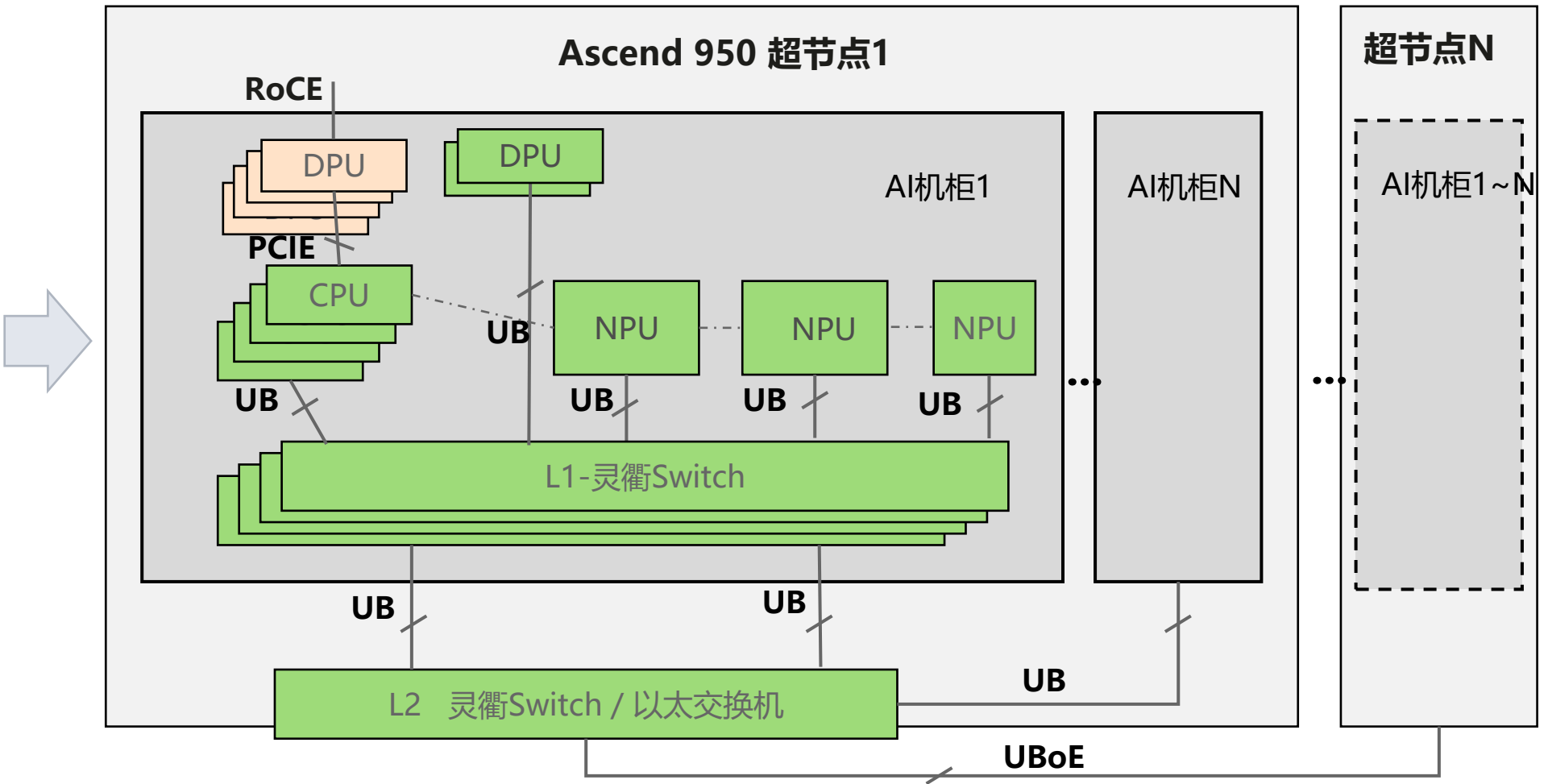
灵衢互联特征 (UB)

对等架构
打破以CPU为中心的互联模式

统一互联协议
UB 统一互联协议，避免协议转换

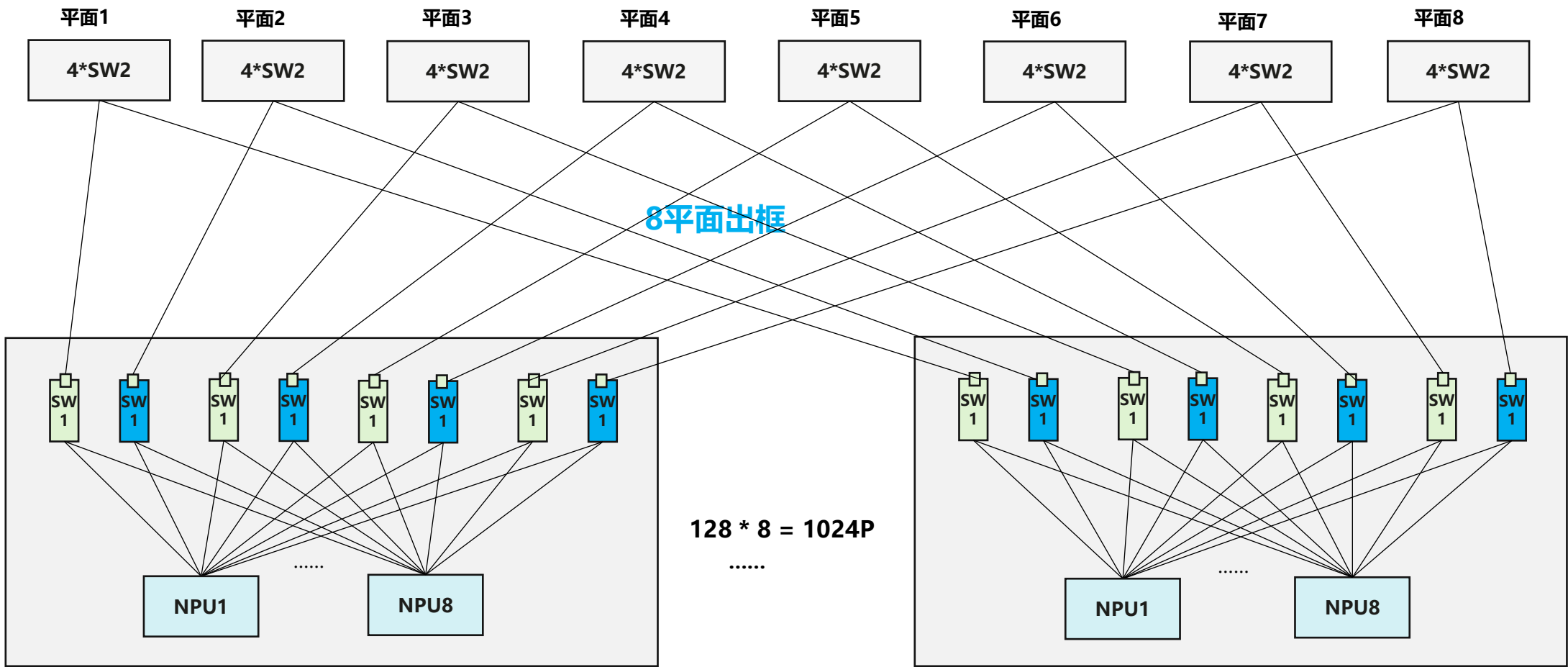
高性能，大规模
低时延，大带宽

全域内存语义
采用内存地址访问，降低编程复杂度



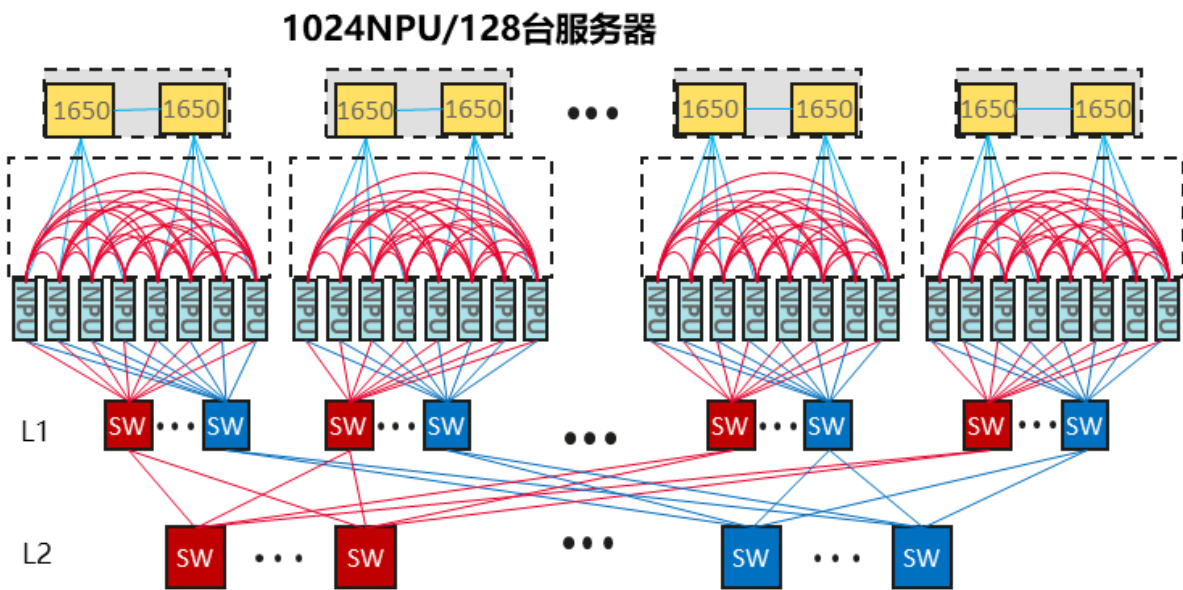
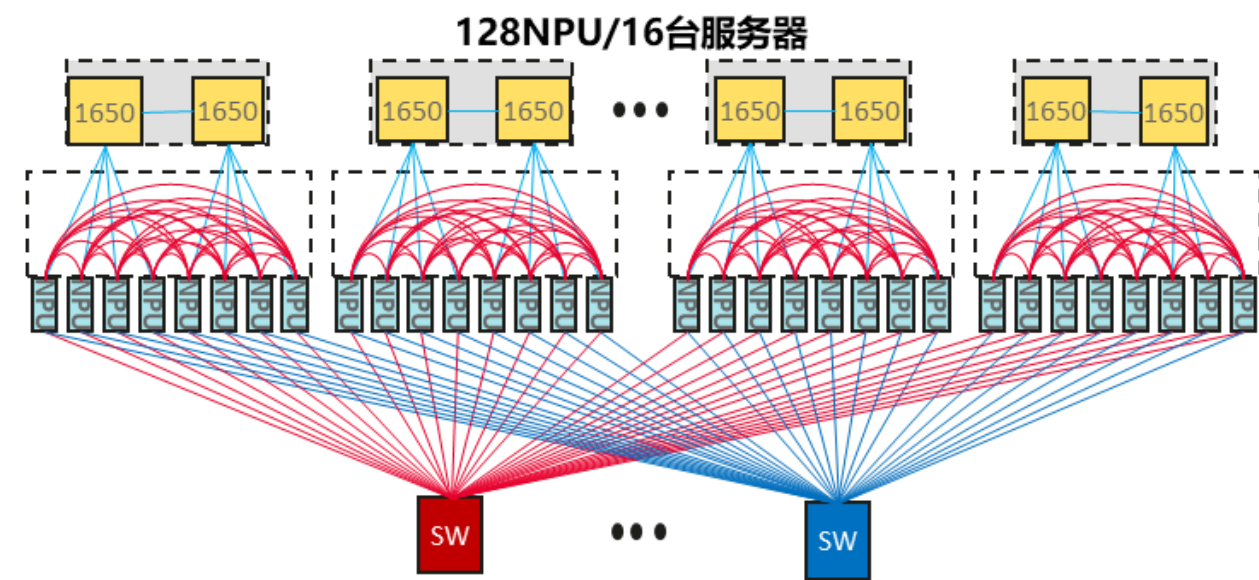
Ascend 950 POD间使用单层SW组网构建1024P超节点集群

- 1、Ascend 950 POD可以采用出框单层SW构建SuperPOD超节点;
- 2、L1 SW和L2 SW构成Clos互联;
- 3、多个独立平面，平面间无互联;



Ascend 950 server使用单层或者两层SW构建多种规格超节点

- 1、128卡超节点：通过一层SW设备实现16台服务器，128张卡的对等全互联；
- 2、1K超节点：通过L1 SW先构建服务器内8张卡接入、L2 SW实现TOR的互联汇聚；
- 3、多个独立平面，平面间无互联；



目录

Part 1 Ascend 950组网架构

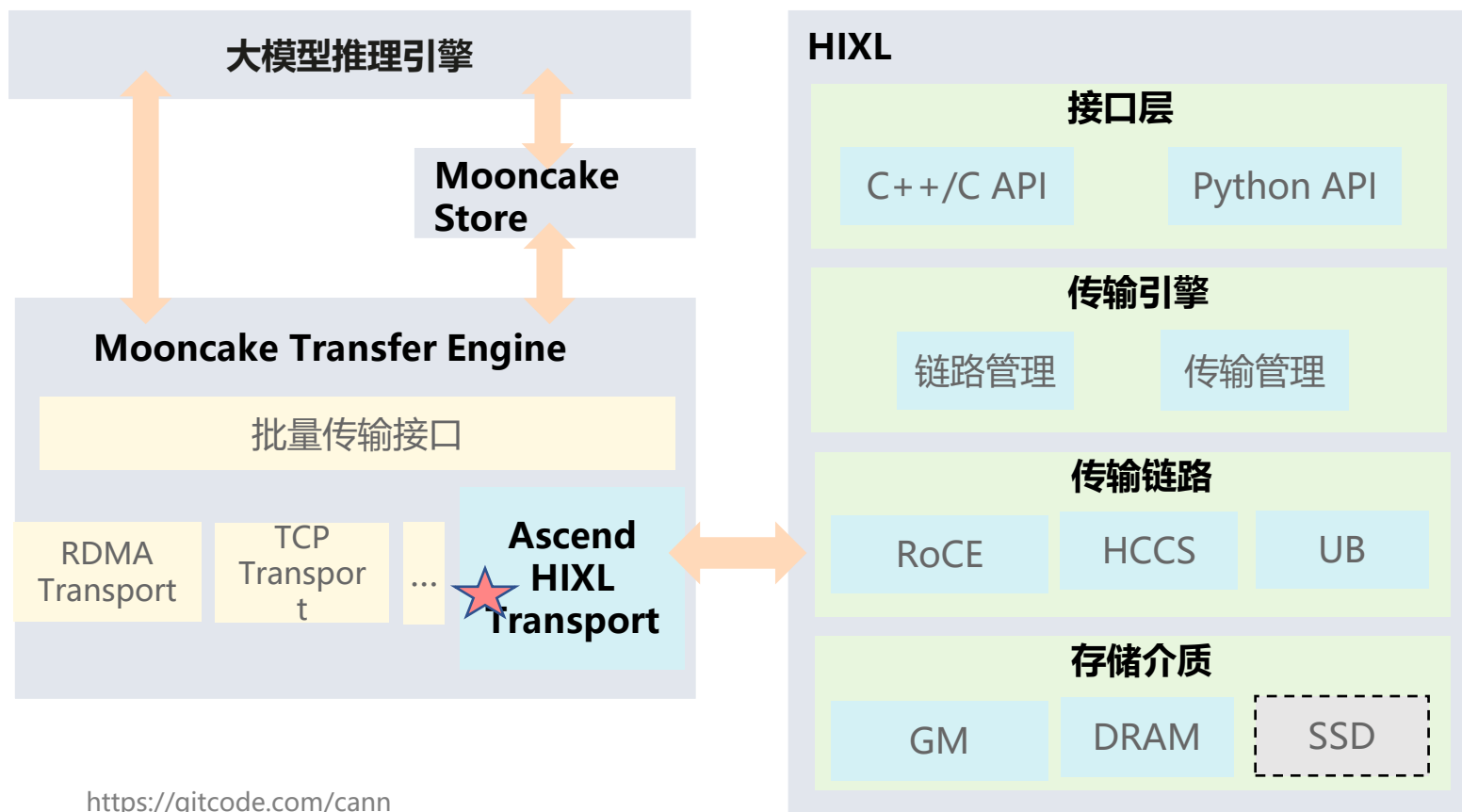
Part 2 HIXL编程API介绍

Part 3 SHMEM AIV直驱URMA

CANN-HIXL：单边通信库，屏蔽硬件差异，提供多方向多链路传输能力

HIXL（Huawei Xfer Library）是CANN提供的**单边点对点通信库**，面向集群场景提供简易、可靠、高效的数据传输能力

- **性能极致**：原生单边通信语义，支持D2rH/H2rD/D2rD/H2rH/等多种单边**零拷贝**数据直传能力，最大化释放硬件性能
- **极简适配**：极简API，屏蔽昇腾芯片代际差异，支持多种网络与灵衢互联（Roce/UB/HCCS等），支持通信资源自动检索与链路智能优选
- **生态协同**：深度联动Mooncake社区，对接vLLM/SGLang等主流推理引擎，为昇腾生态分布式KV Cache提供核心底座



<https://gitcode.com/cann>

引擎初始化/去初始化

Initialize/Finalize

- 自动获取NPU通信资源
- 仅需配置host ip/port作为引擎唯一标识

链路管理

Connect/Disconnect

- 仅需指定对端引擎唯一标识
- 链路异常时支持自动清理
- 动态扩缩容

本地内存注册/注销

RegisterMem/DeregisterMem

远端内存直访

TransferSync

TransferAsync/GetTransferStatus

- 仅需指定对端引擎唯一标识，以及本地、远端读写地址信息

HIXL编程API：分层接口设计，支持多种互联协议

HIXL Engine 提供极简易用、用户友好的单边通信API

HIXL CS封装Client-Server能力，提供Client-Server模式的建链及传输能力

分层 API 使用方式

LAYERED API

Mooncake 接入层

通过编译开关使能 Ascend HIXL 通路

`cmake .. -DUSE_ASCEND=ON`

使能 HIXL 编译方式

HIXL Engine

提供极致易用的用户友好 API

Initiate: 指定本地 ip:port 与唯一 name

RegisterMem: 注册 Host / Device 类型地址

Connect: 指定远端 ip:port 与唯一 name

TransferSync / Async: 指定远端 name、地址和长度

HIXL CS API

面向单边通信场景的 Client / Server 模式接口

Server

HixlCSServerCreate

HixlCSServerRegMem / UnregMem

HixlCSServerListen / Destroy

Client

HixlCSClientCreate / Connect

HixlCSClientRegMem / UnregMem

HixlCSClientGetRemoteMem

HixlCSClientBatchPut/Get Sync/Async

HixlCSClientQueryCompleteStatus

调用流程摘要

ENGINE / CS FLOW

HIXL Engine

Client

- 1 Initiate 本地 name
- 2 RegisterMem Host / Device
- 3 Connect 远端 name
- 4 TransferSync / Async

Server

- 1 Initiate 本地 name
- 2 RegisterMem Host / Device
- 3 等待远端 Connect
- 4 等待传输结束 → DeregMem / Finalize

HIXL CS Client / Server

Client

- 1 ClientCreate → ClientRegMem
- 2 ClientConnect
- 3 ClientGetRemoteMem
- 4 BatchPut/Get Sync/Async

Server

- 1 ServerCreate → ServerRegMem
- 2 ServerListen
- 3 提供远端内存描述
- 4 等待传输 → UnregMem / Destroy

UNIFIED ABSTRACTION

统一接口
抽象

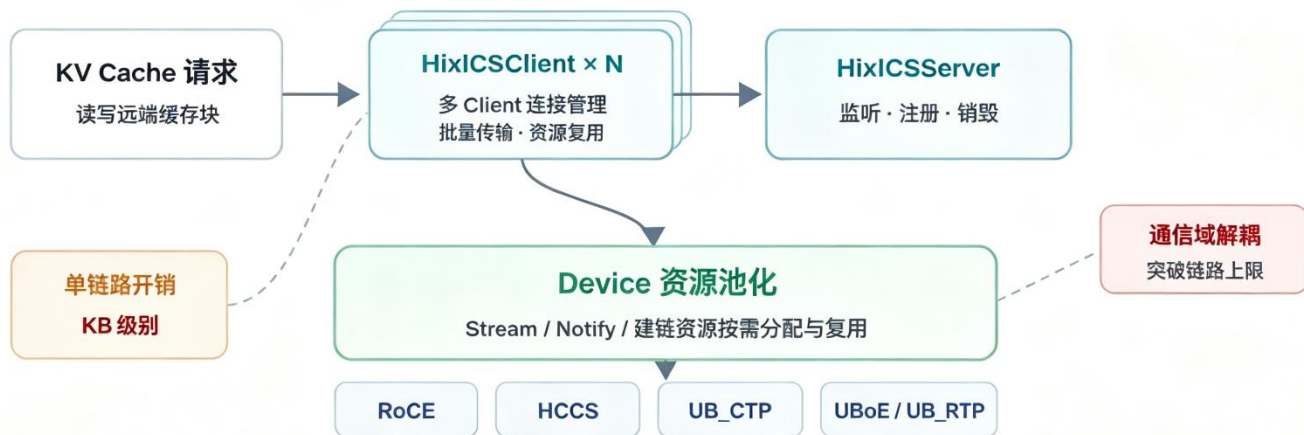
解决上层业务跨协议适配成本高的问题

Mooncake 不直接感知 RDMA / HCCS / UB 等底层差异，统一通过 HIXL Engine 与 HIXL CS API 完成生命周期管理、建链、内存注册、远端内存交换和批量传输；底层协议差异由 HIXL 屏蔽，上层保持稳定调用模型。

HIXL CS架构：解决KV Cache池化关键瓶颈

CS 层资源池化模型

LINK DOMAIN DECOUPLING · DEVICE POOL



面向 KV Cache 池化的收益

KEY IMPACT

链路规模

千卡集群全互联

通信域与链路资源解耦，支撑更大规模扩容。

单链路内存

KB 级别

建联资源池化复用，降低 Device 内存占用。

1

链路规模从设备约束中解放

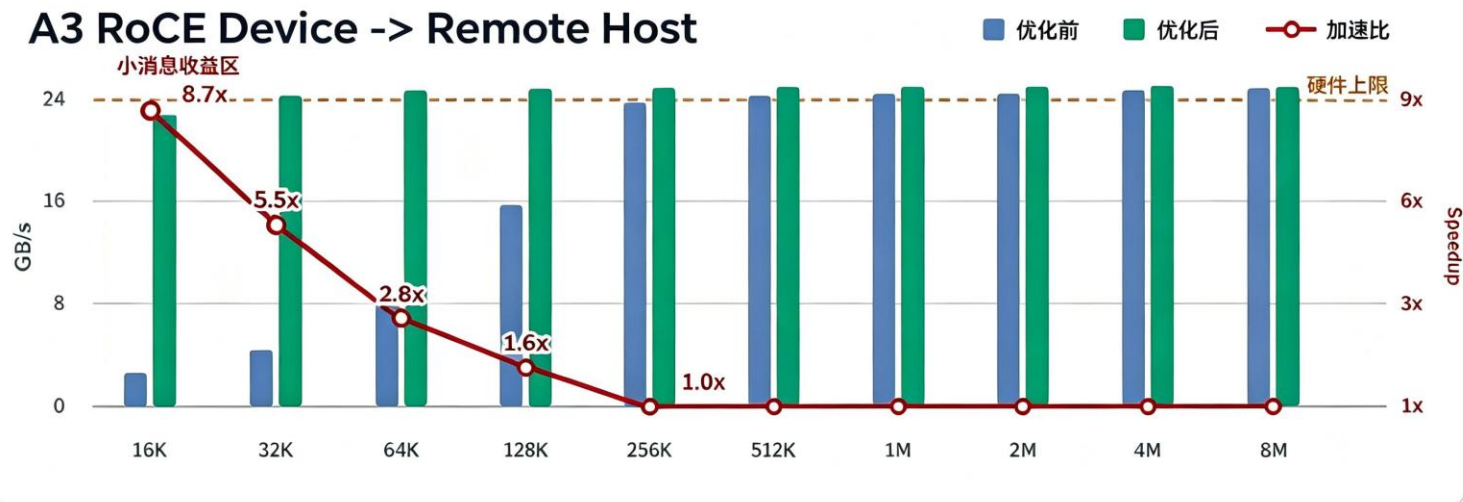
适配 KV Cache 池化中的多节点，多副本和动态路由场景。

2

Device 资源从静态占用变为池化复用

释放被连接资源挤占的显存空间，优先保障缓存容量。

A3 RoCE Device -> Remote Host



小包场景收益极高

16KiB 从 2.48GB/s 提升到 **21.64GB/s**，加速 **8.7x**。

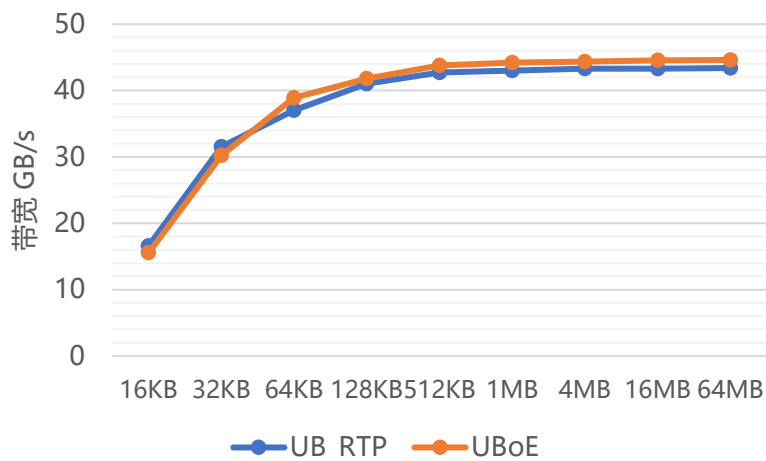
大包性能无损失

256KiB 及以上基本持平，稳定 **1.0x**，保持硬件满带宽。

HIXL: 释放Ascend 950 UB高性能传输能力

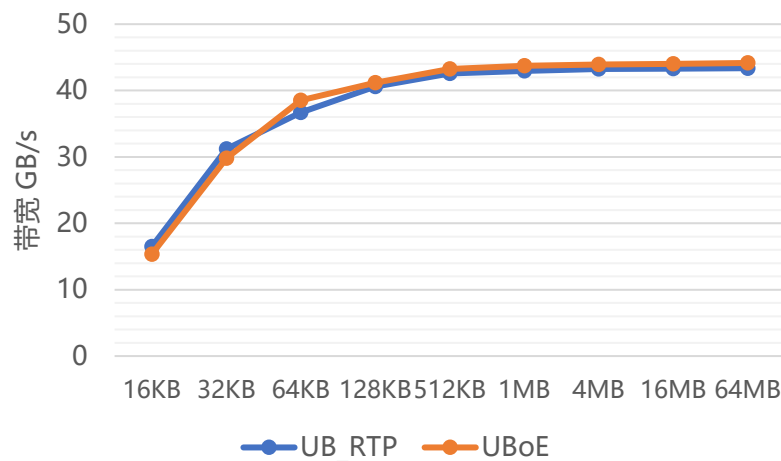
UB_RTP & UBoE Device->Remote Device

带宽实测数据



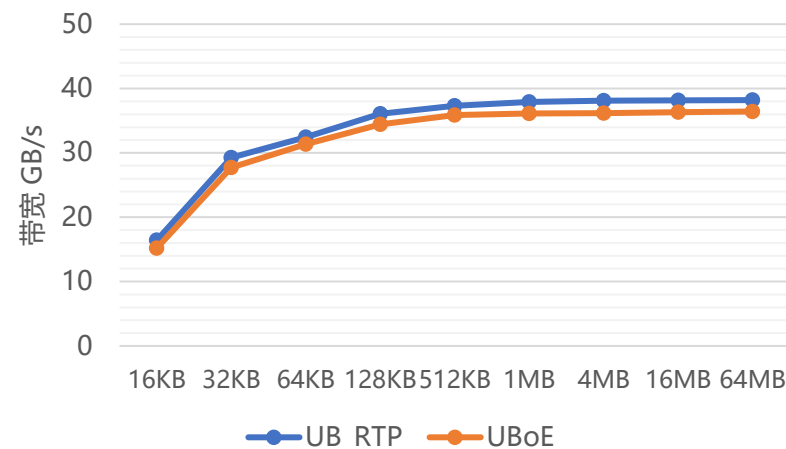
UB_RTP & UBoE Device->Remote Host

带宽实测数据

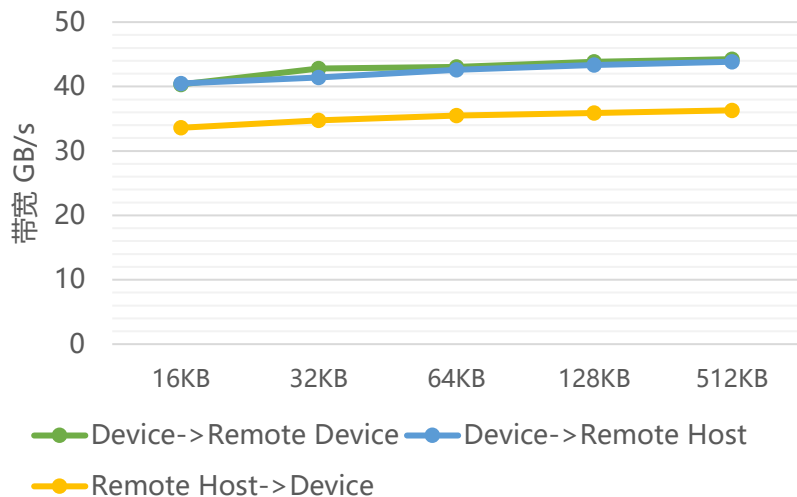


UB_RTP & UBoE Remote Host->Device

带宽实测数据

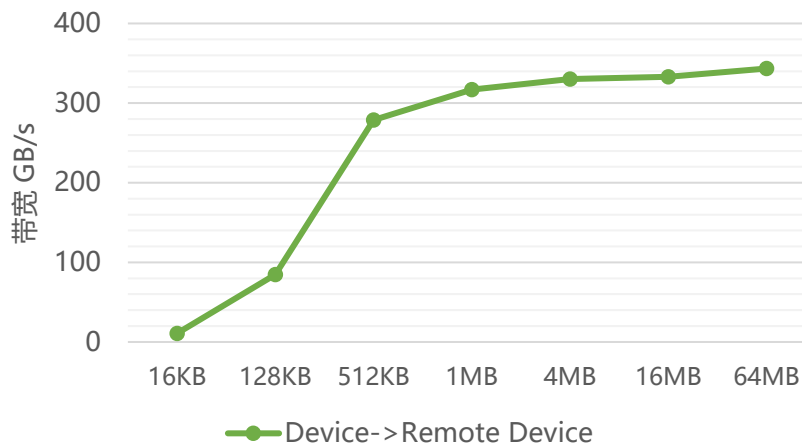


UBoE 4 Channel 并发带宽实测数据



UB_CTP CLOS口 Device->Remote Device

带宽实测数据



- UB_RTP/UBoE 基于 1×X4 UB 接口，当传输数据量≥128KB 时，可基本跑满接口带宽
- 小数据包场景下，可通过多 Channel 并发传输方式充分利用带宽
- UB_CTP 支持 UB 互联端口拓扑可选，可按需选用 FullMesh 或 Clos 拓扑的UB互联端口

更多性能基准用例及详细数据，可参考HIXL社区：
<https://gitcode.com/cann/hixl>

HIXL：异构与跨代际互联

异构互联：

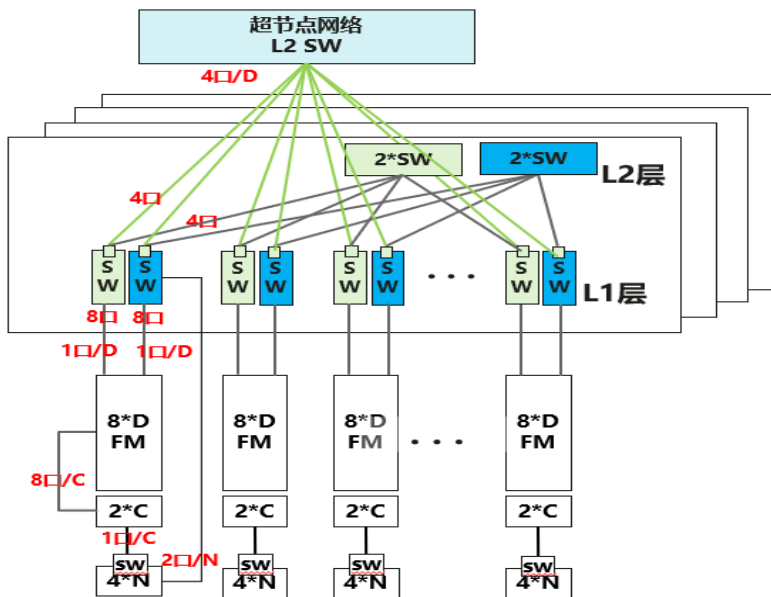
- **PD分离**：实现NPU与GPU/TPU等友商芯片异构互联互通
- **KV池化**：依托内存盘扩容DRAM，构建多级存储能力，增大KV Cache池总容量

跨代际互联：

- 支持昇腾各代际芯片互联互通

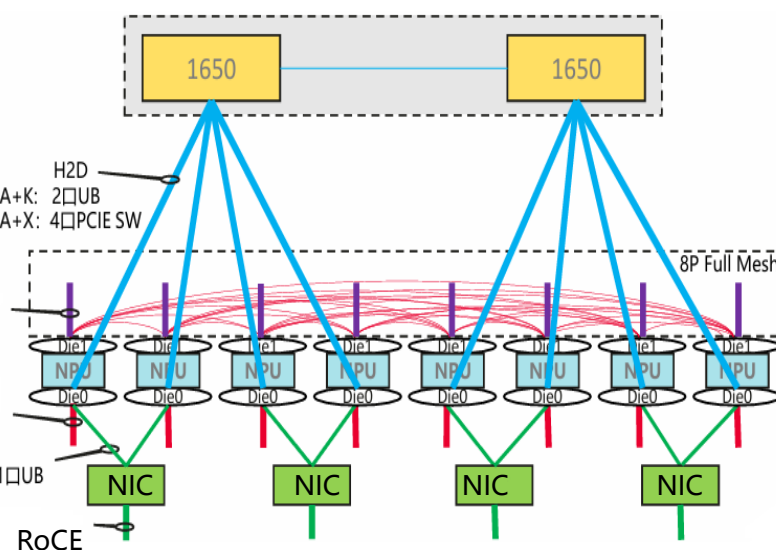


HI XL: 适配Ascend 950全硬件形态



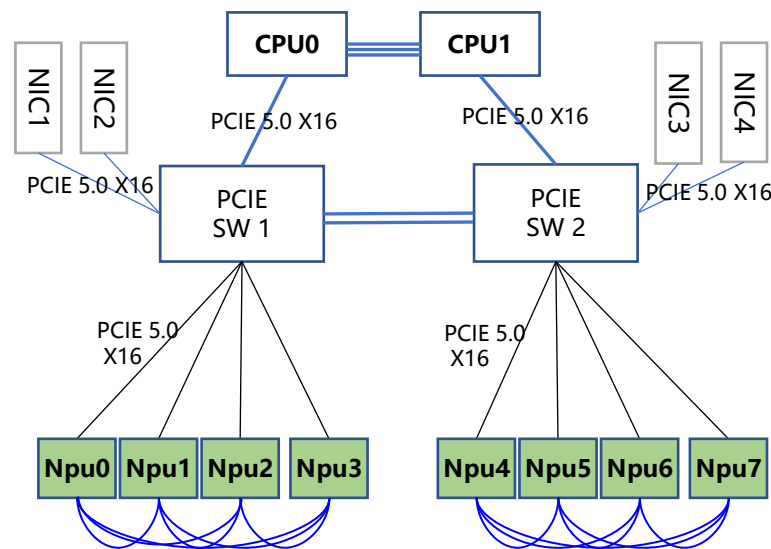
POD

- ① 1D fullmesh+clos拓扑, 64 NPU + 16 CPU
- ② 计算板: 每D出7*X4 UB 端口组成Fullmesh 连接, 每D出8口对接L1 SW
- ③ 交换板: 通过不同的L2 SW配置, 可以等效支持每D出4个端口或者8端口, 作为框内板间互联或者框间互联



Server

- ① 8 NPU + 2 CPU, CPU节点支持鲲鹏及X86
- ② server内: Fullmesh拓扑, 7*X4 UB口
- ③ 出server: Clos拓扑, 8*X4 UB口
- ④ 参数面: 1*X4 UB口



标卡

- ① 基于X86底座，支持四卡UB互联
- ② 每卡3个X4 UB端口组成Fullmesh连接
- ③ 可接第三方网卡，做参数面互联方案

- **硬件全域兼容：**HIXL全面适配Ascend 950全硬件形态，覆盖POD、Server、标卡各类形态
- **通信智能调度：**HIXL自动感知硬件通信资源，智能优选路由，支持多路并发充分释放硬件互联带宽
- **协议灵活可选：**HIXL原生支持UB CTP/UB RTP/UBoE/RoCE多类通信协议，用户可按需自主选择

目录

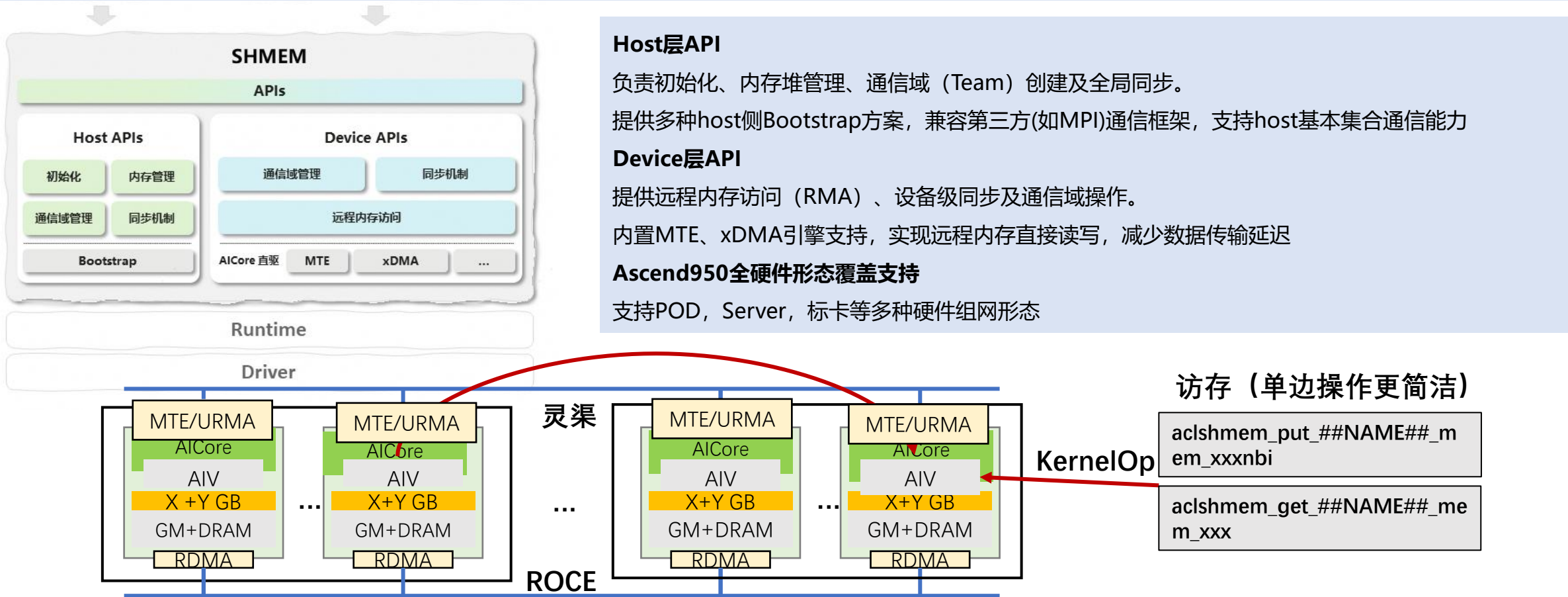
Part 1 Ascend 950组网架构

Part 2 HIXL编程API介绍

Part 3 SHMEM AIV直驱URMA

CANN-SHMEM：基于对称内存的单边通信库

- SHMEM是CANN提供的提供高效、易用的**AIV直驱访存能力**，支撑全局对称内存构建、通算融合算子创新。简化卡间数据同步与通算融合算子开发流程。
- 性能极致：** 内置MTE、xDMA引擎支持，实现远程内存直接读写，减少传输延迟，支持D2H/D2D等多种AIV直驱能力，最大化释放硬件性能
- 极简适配：** 基于OpenShmem标准，极简API，屏蔽昇腾芯片代际差异，支持多种网络与灵衢互联（Roce/UB等）
- 生态协同：** 对接Pytorch，Triton-Dist等三方生态，实现跨框架、跨进程的零拷贝数据交换，显著降低通信开销



什么是 AIV 直驱

01 核心定义

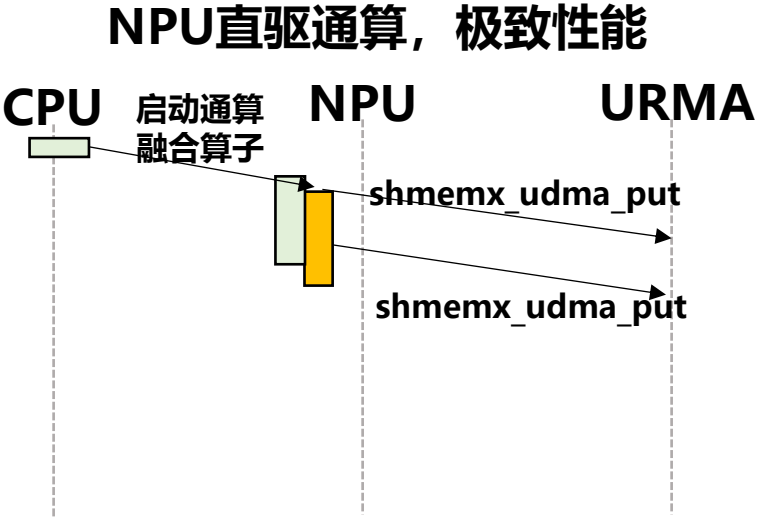
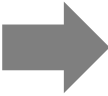
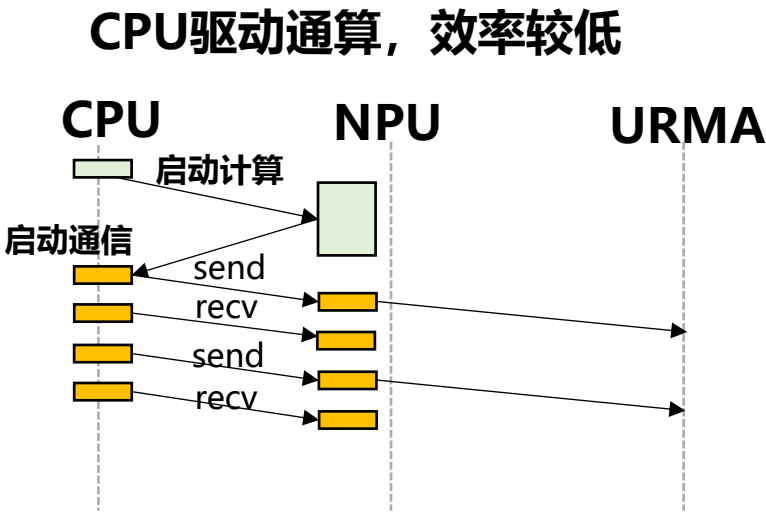
突破传统架构限制，让运行在AIV（Vector Core）上的Kernel直接下发通信任务，绕过Host/AICPU的中间调度环节，实现通信任务的高效处理。

02 关键变化

通信任务提交主体由Host迁移至AIV，优化了控制路径效率；Host仅负责前期的资源创建、设备注册与链路建立，聚焦于宏观控制面管理，释放算力潜力。

03 性能收益

AIV发起通信任务，让计算通信融合更简单，更高效。计算通信的并行，转换成普通CV融合算子的并行，并且达成更低的时延。



SHMEM 与 URMA 的角色定位

SHMEM 提供编程模型，URMA 提供传输语义

01 SHMEM 对称内存通信库

定位：标准兼容的通信接口

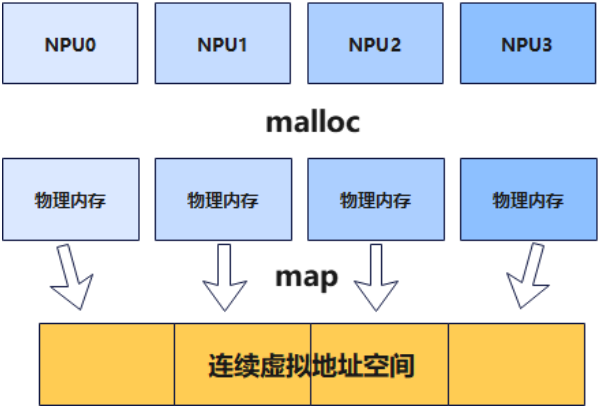
基于 OpenSHMEM 标准，专为昇腾异构架构优化，实现进程间低延迟、高带宽的高效通信编程API。

核心：全局统一地址空间

打破物理节点边界，将多 NPU 的 GM 抽象为共享资源池，支持远端内存的直接读写，无需显式拷贝。

接口：极简的并行编程原语

提供 shmem_put/get 等数据传输接口及 shmem_barrier 同步原语，极大简化了分布式并行程序的开发流程。



全局地址空间模型示意

图示展示了 SHMEM 如何通过全局地址空间抽象，实现跨进程的内存共享与直接访问。这种模型消除了传统通信中繁琐的数据拷贝过程，让分布式内存如同本地内存般易于操作，不仅降低了并行编程的复杂度，更通过硬件级的直接访问大幅提升了数据传输效率，为昇腾集群的高性能计算提供了坚实的通信基础。

02 URMA 统一远程内存访问协议

定位：灵渠总线的底层通信协议

运行于昇腾灵渠总线之上，定义了设备间内存访问的标准交互规则，是硬件层面通信的通用语言。

能力：访问语义与格式归一化

统一本地与远端内存的访问逻辑，标准化 WQE（工作队列元素）格式，降低硬件适配与通信的复杂度。

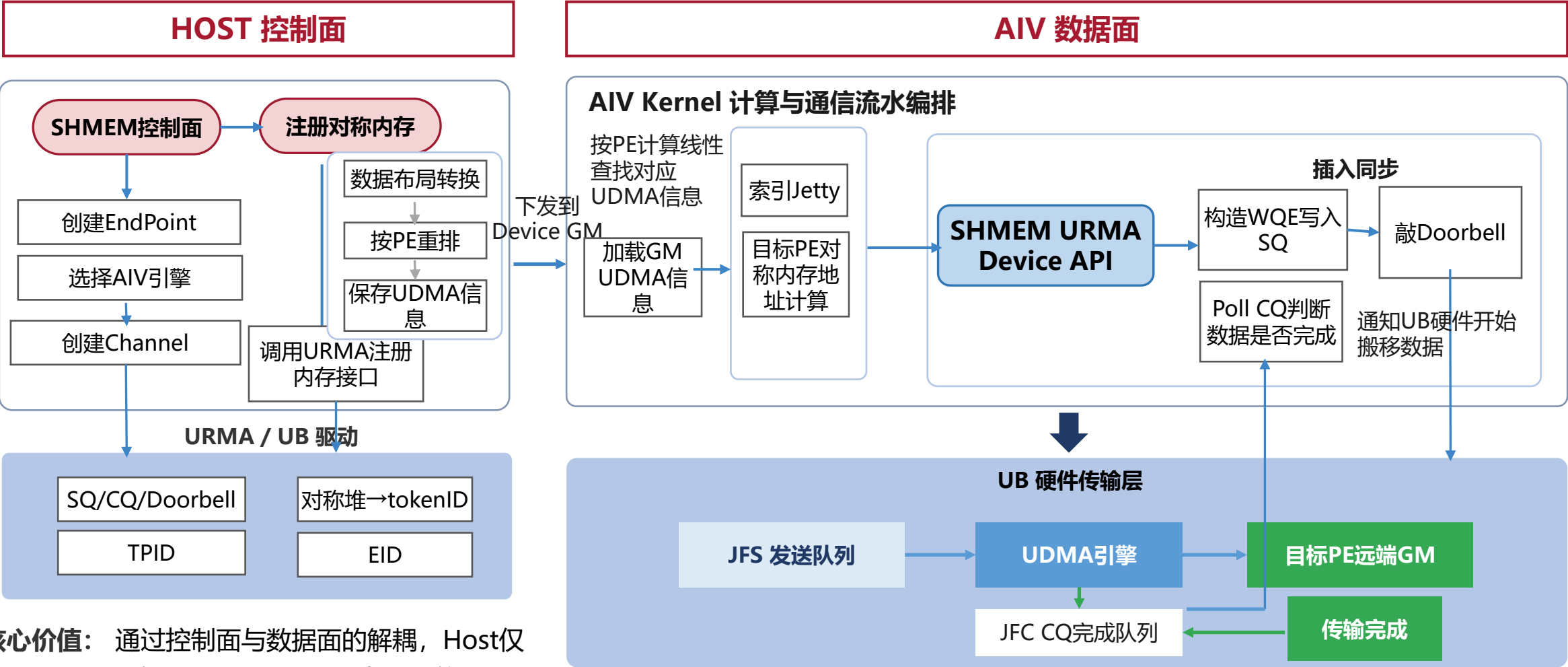
价值：AIV 直驱的技术底座

AIV 通信编程接口遵循 URMA 规范构造请求，是实现芯片级直驱、充分释放硬件算力的关键基础。



SHMEM AIV 直驱 URMA 整体架构

控制面与数据面的职责分离



核心价值： 通过控制面与数据面的解耦，Host仅负责一次性的资源配置与上下文下发，通信数据面操作由AIV硬件引擎接管，实现低时延、高吞吐的极致通信性能。

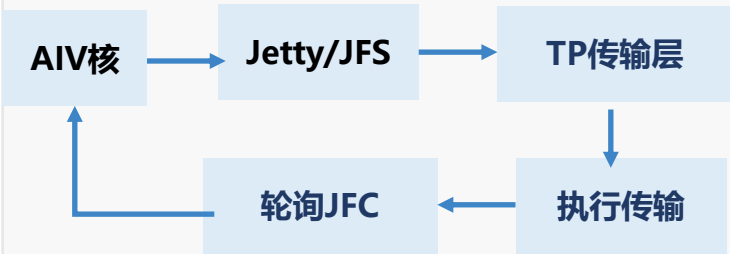


支持灵活提交模式

01 单 Jetty 逐条提交

定位：不支持并发 · 单条提交粒度

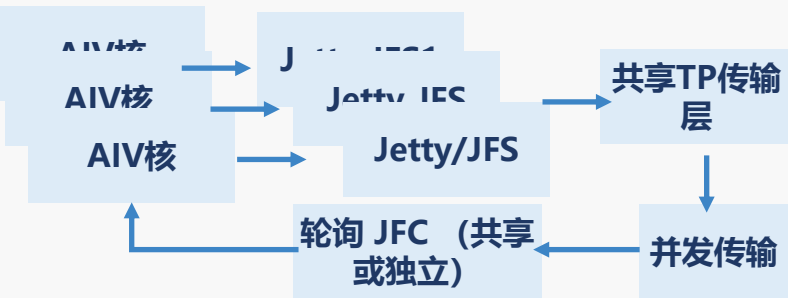
关键接口：
aclshmemx_udma_put/get_nbi



02 多 Jetty 并发提交

定位：多核或者SIMT多线程并发

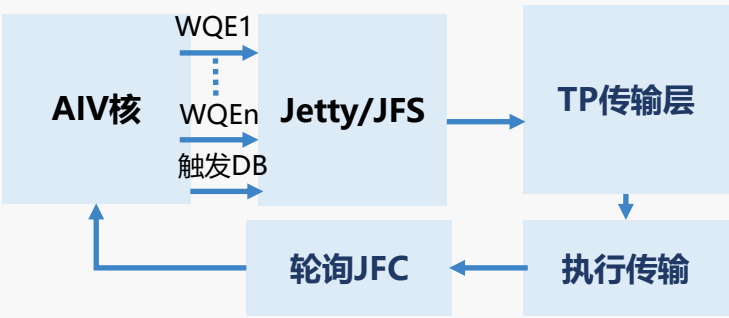
关键接口：
aclshmemx_udma_qp_put/get_nbi



03 聚合提交 (Deferred)

定位：不支持并发 · 批量提交粒度

关键接口： defer / submit 重载接口



核心差异：提交粒度（逐条 vs 批量）

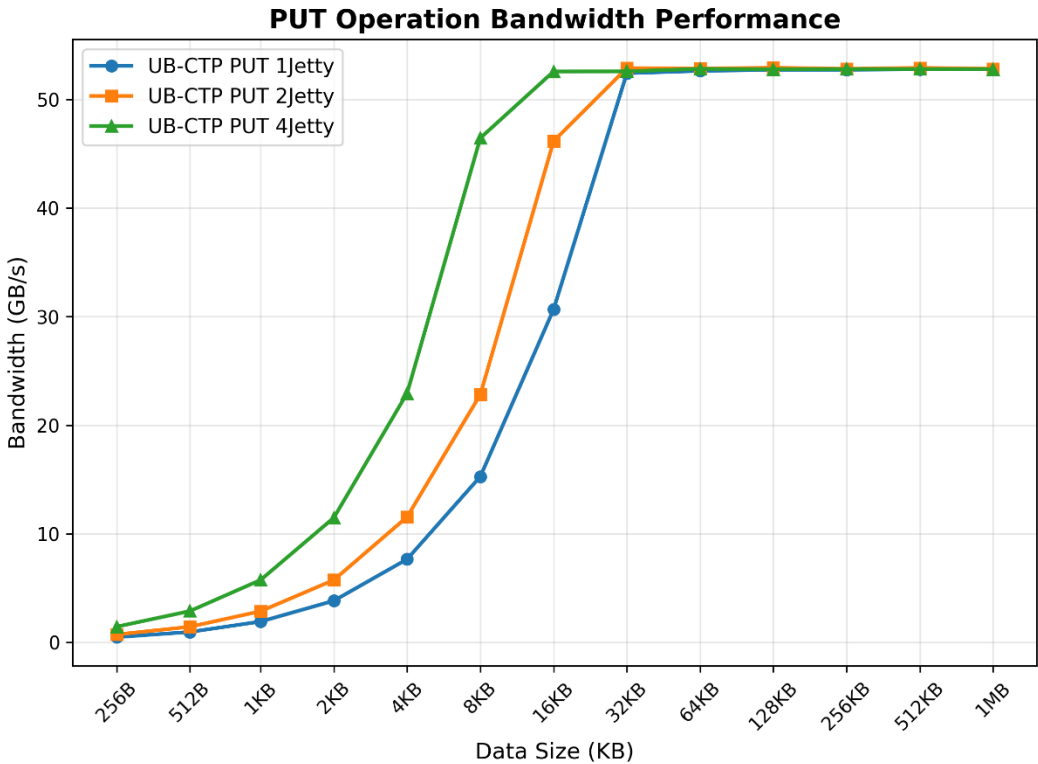
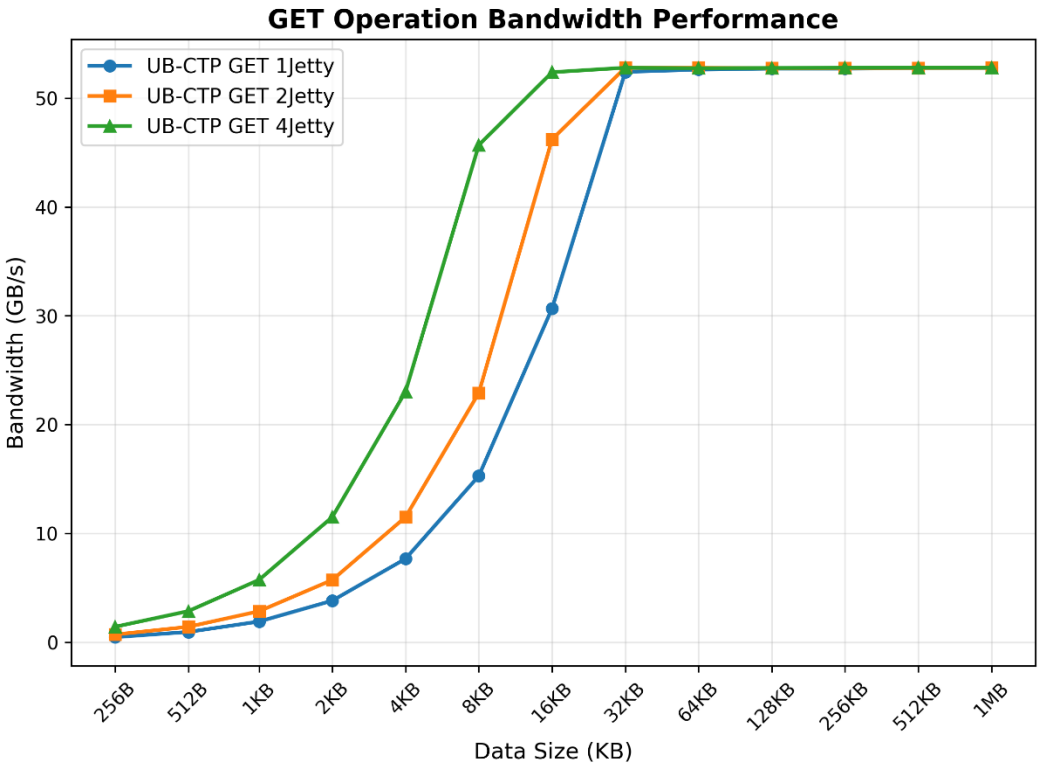
逐条提交以**单请求**为单位独立传输，灵活性高但协议开销占比大；聚合提交将多请求**打包批量**发送，大幅降低交互频次与系统调用开销，提升单位时间内的数据有效载荷，尤其适配小包频发场景。

核心差异：并发维度（单队列 vs 多队列）

单队列模式为**串行处理**，逻辑简单易维护，适合轻载或低时延场景；多队列模式实现**多通道并行**，充分利用多核算力与硬件队列资源，在中小数据量传输场景，最大化释放硬件的并发潜力。

SHMEM AIV直驱URMA性能总览

UB-CTP FULLMESH性能



UB-CTP 单接口下发延时

下发方式	单接口软件下发耗时
逐条提交WQE	<500ns
聚合提交下发	<10ns

更多性能数据请参考：

<https://gitcode.com/cann/shmem/wiki/perftest-udma-mte-d2d-clos-ascend950>

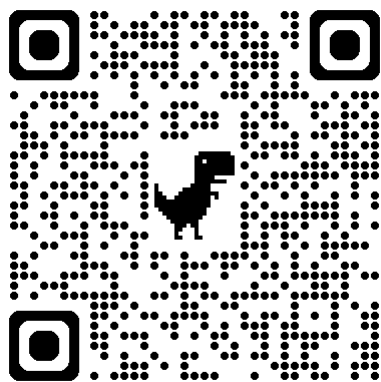
更多URMA使用场景，请参考SHMEM社区样例：

<https://gitcode.com/cann/shmem>



代码仓地址：<https://gitcode.com/cann/shmem>

文档地址：<https://shmem-doc.pages.dev>



代码仓链接



SIG会议订阅链接

HIXL赏金猎人活动~欢迎参与

CANN Be X计划之赏金猎人【秋季】招募开启

活动一、HIXL体验官招募：文档/代码捉虫

活动详情：<https://gitcode.com/cann/hixl/issues/644>



奖品：HUAWEI WATCH Ultimate 2、华为 AI 眼镜、社区定制礼品（晴雨伞）等

活动二、HIXL仓“质量加固”有奖活动

活动详情：<https://gitcode.com/cann/hixl/issues/613>



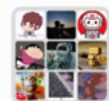
奖品：Huawei手环 11 NFC款

HIXL代码仓：<https://gitcode.com/cann/hixl>

交流咨询：



HIXL小助手



群聊：CANN开源社区HIXL交流群



Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN

HIXL: 异构与跨代际芯片互联架构

